

Eje Temático 2: Electrónica Digital para Control

Introducción

Profundizar en la electrónica digital es clave, porque es el lenguaje que hablan los controladores modernos, desde un simple Arduino hasta los PLCs industriales que van a encontrar en cualquier fábrica.

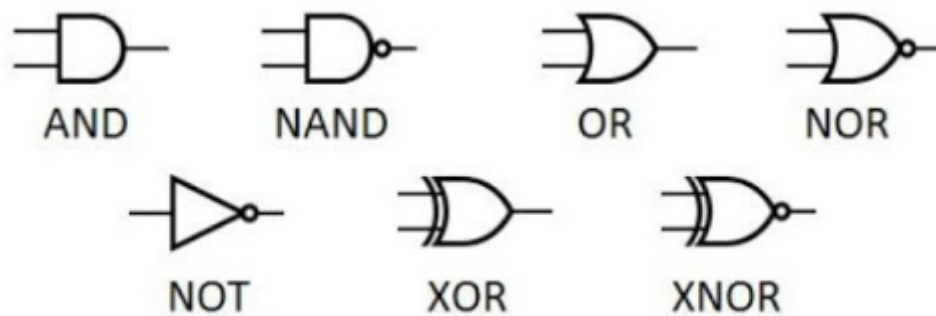
En el eje anterior vimos los conceptos de control y automatización, incluyendo los lazos abierto y cerrado. Ahora, nos meteremos de lleno en la tecnología que hace posible implementar la "inteligencia" de esos sistemas: la **electrónica digital**. Mientras que la electrónica analógica trabaja con señales que varían continuamente, la digital se basa en dos únicos estados: ALTO y BAJO, representados por los números binarios 1 y 0. Entender cómo generar, manipular y almacenar estos "unos" y "ceros" es fundamental para comprender cómo funcionan los microcontroladores, PLCs y computadoras que gobiernan los procesos electromecánicos modernos. Este apunte es un repaso y profundización de estos temas cruciales.

1. El Interruptor Perfecto y los Ladrillos Lógicos

- **Transistor en Conmutación: El Corazón del Bit (Repaso Profundo)**

- *¿Por qué transistores?* En el mundo digital, necesitamos elementos que puedan estar en dos estados bien definidos (como un interruptor abierto o cerrado) y que puedan cambiar entre ellos muy rápidamente y millones de veces sin desgastarse. Los interruptores mecánicos son lentos y se gastan. Los transistores (tanto BJT como MOSFET) son interruptores electrónicos casi ideales para esto.
- *¿Cómo funcionan como interruptor?*
 - **Corte:** Cuando no aplicamos la señal de control adecuada (corriente de base $I_b \approx 0$ para BJT, o tensión Puerta-Surtidor V_{GS} por debajo del umbral para MOSFET), el transistor se comporta como un interruptor **abierto**. Prácticamente no circula corriente entre Colector y Emisor ($I_c \approx 0$) o Drenador y Surtidor ($I_d \approx 0$), y la tensión entre estos terminales es alta ($V_{CE} \approx V_{cc}$ / $V_{DS} \approx V_{dd}$). Este estado lo asociamos al **0 lógico** o nivel BAJO.
 - **Saturación:** Cuando aplicamos una señal de control suficiente (I_b adecuada para BJT, V_{GS} bien por encima del umbral para MOSFET), el transistor se comporta como un interruptor **cerrado**. Permite que circule la máxima corriente posible (limitada por el circuito externo) entre C-E (I_c) o D-S (I_d), y la tensión entre estos terminales cae a un valor muy bajo ($V_{CEsat} \approx 0.2V$ / $V_{DSon} \approx$ muy bajo). Este estado lo asociamos al **1 lógico** o nivel ALTO.
- *Niveles Lógicos y Familias:* En la práctica, los 0s y 1s se representan con rangos de voltaje. Por ejemplo, en la familia TTL (Transistor-Transistor Logic, más antigua), un '0' podía ser de 0 a 0.8V y un '1' de 2V a 5V. En CMOS (Complementary Metal-Oxide-Semiconductor, la más usada hoy), los rangos suelen ser más cercanos a los extremos de la alimentación (ej: 0V para '0', 3.3V o 5V para '1'). *¿Por qué CMOS?* Principalmente por su bajísimo consumo de energía en estado estático.

- **Compuertas Lógicas: Tomando Decisiones Binarias**



- *¿Por qué compuertas?* Son los circuitos electrónicos básicos que realizan operaciones lógicas fundamentales sobre las señales binarias (0s y 1s). Son los "ladrillos" con los que se construye cualquier circuito digital complejo. Cada compuerta implementa una función booleana específica.
- **NOT (Inversor):** La más simple. Su salida es siempre el estado opuesto a la entrada. *Cómo:* Un simple transistor en configuración de emisor común puede actuar como inversor. *Función:* Negación. *Símbolo:* Triángulo con un círculo en la salida. *Tabla de Verdad:* Si entra 0, sale 1; si entra 1, sale 0. *Ecuación:* $Y = \bar{A}$ (se lee "A negada").
- **AND:** La salida es 1 *sólo si todas* sus entradas son 1. Si alguna entrada es 0, la salida es 0. *Cómo:* Conceptualmente, como interruptores en serie; todos deben estar cerrados (1) para que pase la señal. *Función:* Producto lógico. *Símbolo:* Forma de "D". *Tabla (2 entradas):* $0 \cdot 0 = 0$, $0 \cdot 1 = 0$, $1 \cdot 0 = 0$, $1 \cdot 1 = 1$. *Ecuación:* $Y = A \cdot B$ (o $Y = AB$). *Aplicación Típica:* Habilitar una acción sólo si se cumplen múltiples condiciones (ej: motor arranca si Pulsador_ON=1 Y Puerta_Cerrada=1).
- **OR:** La salida es 1 *si al menos una* de sus entradas es 1. Sólo es 0 si todas las entradas son 0. *Cómo:* Conceptualmente, como interruptores en paralelo; basta que uno esté cerrado (1) para que pase la señal. *Función:* Suma lógica. *Símbolo:* Forma curva con punta. *Tabla (2 entradas):* $0+0=0$, $0+1=1$, $1+0=1$, $1+1=1$. *Ecuación:* $Y = A + B$. *Aplicación Típica:* Activar una alarma si se dispara Sensor_A=1 O Sensor_B=1.
- **NAND (NOT-AND):** Es una AND seguida de una NOT. La salida es 0 *sólo si todas* las entradas son 1 (es la inversa de la AND). *Símbolo:* Como la AND pero con círculo en la salida. *Tabla:* Inversa de la AND. *Ecuación:* $Y = \neg(A \cdot B)$.
- **NOR (NOT-OR):** Es una OR seguida de una NOT. La salida es 1 *sólo si todas* las entradas son 0 (es la inversa de la OR). *Símbolo:* Como la OR pero con círculo en la salida. *Tabla:* Inversa de la OR. *Ecuación:* $Y = \neg(A+B)$.
 - *¿Por qué NAND y NOR son "Universales"?* Porque combinando sólo compuertas NAND (o sólo compuertas NOR) se puede construir cualquier otra función lógica (NOT, AND, OR, XOR). Esto es muy importante en la fabricación de circuitos integrados.
- **XOR (OR Exclusiva):** La salida es 1 *sólo si las entradas son diferentes*. Si son iguales (ambas 0 o ambas 1), la salida es 0. *Símbolo:* Como la OR pero con una línea curva adicional en la entrada. *Tabla (2 entradas):* $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. *Ecuación:* $Y = A \oplus B = A \neg B + \neg A B$. *Aplicación Típica:* Comparador de bits, generador de paridad, semi-sumador binario.

- **XNOR (NOT-XOR o Coincidencia):** La salida es 1 sólo si las entradas son iguales. Es la inversa de la XOR. *Símbolo:* Como la XOR pero con círculo en la salida. *Tabla:* Inversa de la XOR. *Ecuación:* $Y = \neg(A \oplus B) = AB + \neg A \neg B$.

Entradas			Operación					
A	B	C	AND	NAND	OR	NOR	XOR	XNOR
0	0	0	0	1	0	1	0	1
0	0	1	0	1	1	0	1	0
0	1	0	0	1	1	0	1	0
0	1	1	0	1	1	0	0	1
1	0	0	0	1	1	0	1	0
1	0	1	0	1	1	0	0	1
1	1	0	0	1	1	0	0	1
1	1	1	1	0	1	0	1	0

2. El Lenguaje de la Lógica: Álgebra de Boole

- *¿Por qué un Álgebra?* George Boole desarrolló este sistema matemático en el siglo XIX para trabajar con la lógica proposicional (verdadero/falso). Resulta que es perfectamente aplicable a los circuitos digitales (1/0). Nos da las herramientas para:
 - Describir la función de un circuito lógico mediante una ecuación.
 - Simplificar circuitos complejos para hacerlos más eficientes.
 - Analizar y diseñar nuevos circuitos lógicos.
- **Postulados y Propiedades Fundamentales:** Son las reglas básicas del juego. Algunas clave (¡hay que manejarlas bien!):
 - Identidad: $A+0 = A$; $A \cdot 1 = A$
 - Anulación: $A+1 = 1$; $A \cdot 0 = 0$
 - Idempotencia: $A+A = A$; $A \cdot A = A$
 - Involución: $\neg(\neg A) = A$ (Negar dos veces es afirmar)
 - Conmutativa: $A+B = B+A$; $A \cdot B = B \cdot A$
 - Asociativa: $A+(B+C) = (A+B)+C$; $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
 - Distributiva: $A \cdot (B+C) = A \cdot B + A \cdot C$; $A+(B \cdot C) = (A+B) \cdot (A+C)$ (¡Ojo con la segunda!)
 - Absorción: $A+(A \cdot B) = A$; $A \cdot (A+B) = A$ (Cómo ayuda: Si 'A' ya es 1, la primera suma da 1; si 'A' es 0, el producto $A \cdot B$ es 0 y la suma da 0. Siempre da 'A').
- **Teoremas de DeMorgan (¡Fundamentales!):** Permiten convertir sumas en productos y viceversa, introduciendo negaciones.
 - Teorema 1: $\neg(A + B) = \neg A \cdot \neg B$ (La negación de una OR es la AND de las negaciones).
 - Teorema 2: $\neg(A \cdot B) = \neg A + \neg B$ (La negación de una AND es la OR de las negaciones).
 - *Cómo usarlos:* Son la clave para pasar de expresiones con AND/OR a expresiones solo con NAND o solo con NOR (aplicando doble negación y DeMorgan). También simplifican mucho expresiones negadas complejas. ("Romper la barra de negación larga y cambiar el signo de la operación que estaba debajo").
- **Funciones Lógicas y Tablas de Verdad:**
 - Una tabla de verdad define *exactamente* qué salida debe tener un circuito para cada posible combinación de entradas.
 - *Cómo obtener la ecuación desde la tabla (Formas Canónicas):*
 - **Suma de Productos (SOP - Sum Of Products):** Nos fijamos en las filas donde la salida (Y) es '1'. Para cada una de esas filas, escribimos un término producto

(un "minitérmino") que valga '1' sólo para esa combinación de entradas (usamos la variable si es '1', la negamos si es '0'). La función final es la suma (OR) de todos esos minitérminos. *Por qué:* Asegura que la función valga '1' exactamente donde dice la tabla.

- **Producto de Sumas (POS - Product Of Sums):** Nos fijamos en las filas donde la salida (Y) es '0'. Para cada una, escribimos un término suma (un "maxitérmino") que valga '0' sólo para esa combinación (usamos la variable negada si es '1', sin negar si es '0'). La función final es el producto (AND) de todos esos maxitérminos. *Por qué:* Asegura que la función valga '0' exactamente donde dice la tabla. A veces, simplificar la POS es más fácil que la SOP.

- **Simplificación de Funciones Lógicas:**

- *¿Por qué simplificar?* Un circuito más simple usa menos compuertas, lo que significa: Menor costo, menor tamaño físico, menor consumo de energía, y menor retardo de propagación (más rápido). ¡Todo ventajas!
- **Simplificación Algebraica:** *Cómo:* Aplicar sistemáticamente los postulados, propiedades y teoremas (especialmente Absorción y DeMorgan) a la ecuación booleana hasta que no se pueda reducir más. Requiere práctica y "ver" qué teorema aplicar.
- **Mapas de Karnaugh (Método Gráfico):** *Por qué:* Para funciones de 3 o 4 variables (o más, pero se complica), el método algebraico puede ser tedioso y es fácil equivocarse u omitir una simplificación. Los Mapas K son una forma visual y sistemática.
 - *Cómo Construir:* Se crea una tabla especial donde las combinaciones de entrada se ordenan de forma que sólo cambie un bit entre celdas adyacentes (Código Gray). Se vuelcan los '1' (o los '0' si simplificamos POS) de la tabla de verdad en el mapa.
 - *Cómo Agrupar:* Se buscan grupos de '1's adyacentes (horizontal, vertical, y los bordes se consideran adyacentes como si el mapa fuera un toroide). Los grupos deben ser de tamaño potencia de 2 (1, 2, 4, 8...) y lo más grandes posible. Un '1' puede pertenecer a varios grupos.
 - *Cómo Obtener la Expresión:* Cada grupo corresponde a un término producto simplificado. Dentro de un grupo, sólo permanecen las variables que *no cambian* de valor (si una variable es 0 y 1 dentro del mismo grupo, se elimina). La función final es la suma (OR) de los términos de cada grupo.

3. Guardando el Estado: Lógica Secuencial y Flip-Flops

- *¿Por qué la Lógica Secuencial?* Los circuitos combinacionales (compuertas solas) tienen una salida que depende *únicamente* de las entradas en ese instante. No tienen memoria. Pero muchos sistemas necesitan "recordar" eventos pasados para decidir qué hacer ahora (ej: un contador, una secuencia de pasos, saber si un botón fue presionado). La lógica secuencial introduce el concepto de **estado interno**, y su salida depende tanto de las entradas actuales como del estado anterior. *Cómo:* Se logra mediante elementos de memoria básicos llamados Flip-Flops.
- **Latch RS (El Pestillo Básico):**
 - *Cómo:* Se construye con dos compuertas NAND o NOR realimentadas (la salida de una a la entrada de la otra). Tiene dos entradas, Set (S) y Reset (R), y dos salidas complementarias Q y $\neg Q$.

- **Funcionamiento (ej: con NOR):** Si $S=1$ y $R=0 \rightarrow Q=1$ (Set, memoriza un '1'). Si $S=0$ y $R=1 \rightarrow Q=0$ (Reset, memoriza un '0'). Si $S=0$ y $R=0 \rightarrow$ Mantiene el estado anterior (¡Aquí está la memoria!).
- **Problema:** Si $S=1$ y $R=1$ (estado prohibido para Latch NOR), ambas salidas intentarían ser 0, lo cual es inconsistente. Si las entradas vuelven a $S=0$ $R=0$ simultáneamente desde el estado prohibido, la salida final es impredecible (condición de carrera). *Por qué es un problema:* Falta de fiabilidad.
- **Asíncrono:** Reacciona a las entradas S y R inmediatamente, sin esperar una señal de sincronismo.

Latch con compuerta NAND

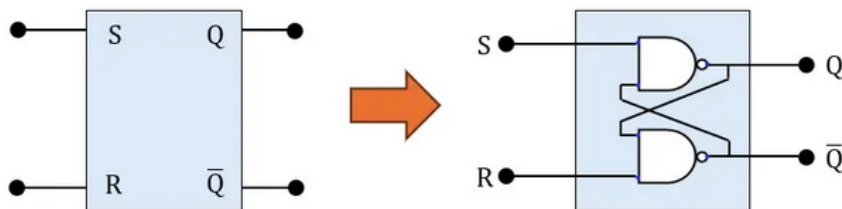


Tabla de Verdad del Latch SR con Compuertas NAND:

S (Set)	R (Reset)	Salida (Q)	Salida ($\bar{Q}$)
1	1	Sin cambio	Sin Cambio
0	1	1	0
1	0	0	1
0	0	Indeterminado	Indeterminado

Latch con compuerta NOR

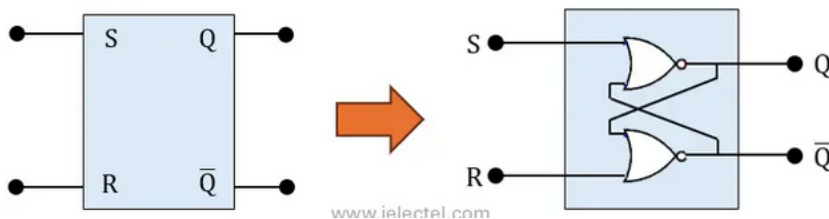


Tabla de Verdad del Latch SR con Compuertas NOR:

S (Set)	R (Reset)	Salida (Q)	Salida ($\bar{Q}$)
0	0	Sin cambio	Sin cambio
1	0	1	0
0	1	0	1
1	1	Indeterminado	Indeterminado

● La Señal de Reloj (Clock): El Director de Orquesta:

- **Por qué:** En sistemas digitales complejos con muchos elementos de memoria, es crucial que todos cambien de estado de forma coordinada. La señal de reloj (Clock, CLK) es una señal periódica (onda cuadrada) que sincroniza las operaciones. Los cambios en los Flip-Flops síncronicos ocurren *sólo* en un instante específico del ciclo de reloj.

- **Flancos:** Lo más común es que los FFs reaccionen al **flanco de subida** (transición de 0 a 1) o al **flanco de bajada** (transición de 1 a 0) de la señal de reloj. Esto asegura un momento preciso para la captura de datos y el cambio de estado.
- **Flip-Flops Sincrónicos (El Corazón de la Memoria Digital):** Son la base de registros, contadores y memorias. Reaccionan a sus entradas de control (J, K, D, T) pero sólo cambian su salida Q en el flanco activo del reloj.
 - **Flip-Flop JK:** El más versátil. Resuelve el problema del RS.
 - **Entradas:** J (similar a Set), K (similar a Reset), CLK.
 - **Funcionamiento (en flanco activo):**
 - J=0, K=0: Mantiene estado anterior (Memoria).
 - J=1, K=0: Q se pone a 1 (Set).
 - J=0, K=1: Q se pone a 0 (Reset).
 - J=1, K=1: Q cambia al estado opuesto (Toggle/Bascula).
 - **Tabla de Excitación:** Dice qué valores deben tener J y K para lograr una transición deseada de Q(t) a Q(t+1). Útil para diseño de contadores/secuenciadores.
 - **Diagrama de Tiempos:** Esencial para visualizar cómo Q cambia en los flancos de CLK según J y K.

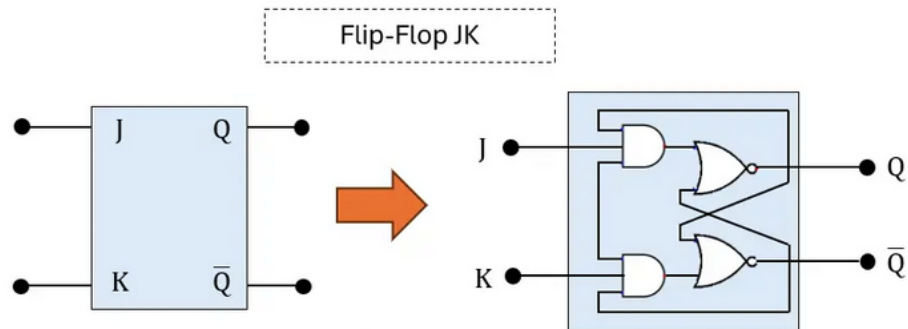


Tabla de Verdad del Flip-Flop JK

J (Set)	K (Reset)	Salida (Q)	Salida ($\bar{Q}$)
0	0	Sin Cambio	Sin Cambio
0	1	0	1
1	0	1	0
1	1	Conmutación	Conmutación

- **Flip-Flop D (Data/Delay):** El más simple para almacenar datos.
 - **Entradas:** D (Dato), CLK.
 - **Funcionamiento:** En el flanco activo del CLK, la salida Q toma el valor que tenga la entrada D en ese instante. Lo "recuerda" hasta el próximo flanco. **Cómo:** Internamente suele ser un JK con J=D y K= $\neg$ D.
 - **Por qué es ideal para registros:** Permite cargar un bit de datos directamente en el FF.
 - **Diagrama de Tiempos:** Q "sigue" a D pero sólo cambia en los flancos.

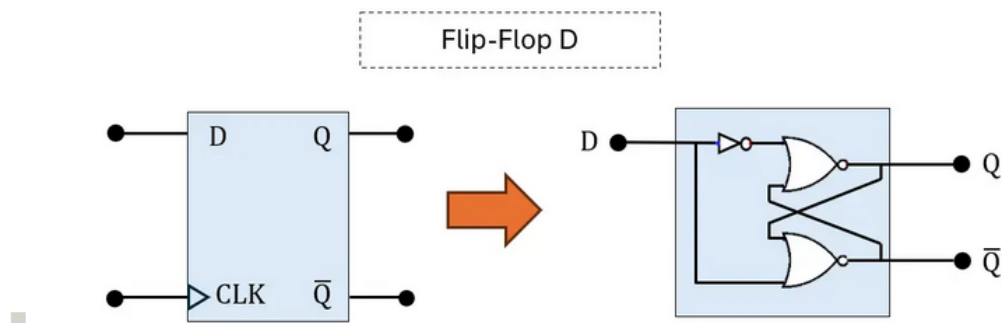


Tabla de Verdad del Flip-Flop D (Data)

D (Data)	Salida (Q)	Salida ($\bar{Q}$)
0	0	1
1	1	0

- **Flip-Flop T (Toggle):** Ideal para contar.
 - *Entradas:* T (Toggle), CLK.
 - *Funcionamiento:* Si T=0, Q mantiene su estado en el flanco. Si T=1, Q báscula (cambia al estado opuesto) en el flanco activo. *Cómo:* Internamente es un JK con J=K=T.
 - *Por qué es útil para contadores:* Si T se mantiene en 1, la salida Q será una onda cuadrada con exactamente la mitad de la frecuencia del CLK (divisor de frecuencia por 2). Conectando varios FFs T en cascada se construyen contadores binarios.
 - *Diagrama de Tiempos:* Muestra la división de frecuencia.

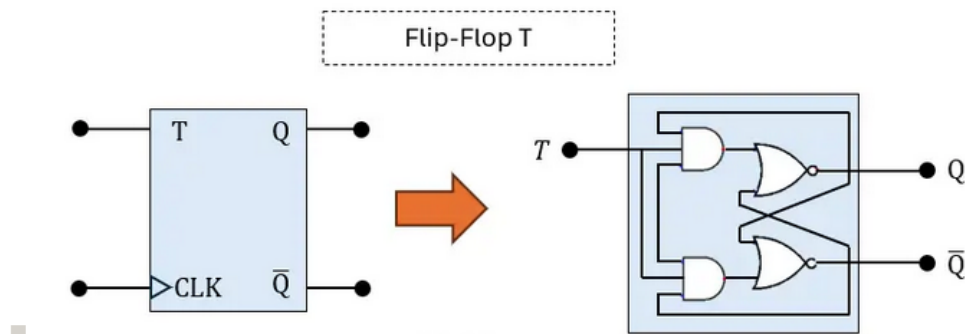


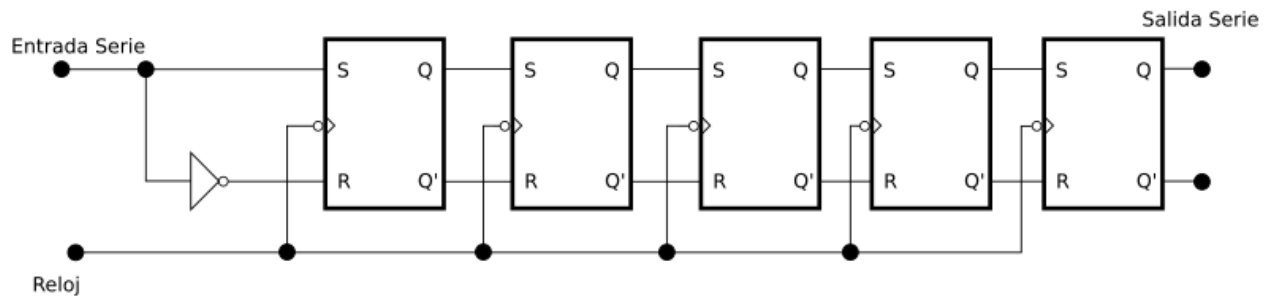
Tabla de Verdad del Flip-Flop T (Toggle)

T	Salida (Q)
0	Memoriza
1	Conmutación

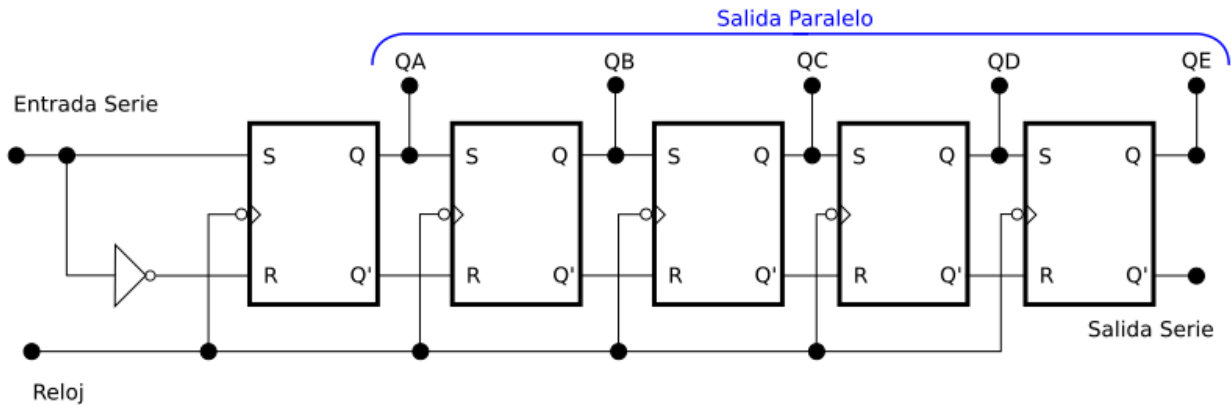
4. Almacenando y Moviendo Información: Registros y Memorias

- **Registros: Guardando Palabras Binarias:**
 - *Cómo:* Un registro de N bits se forma simplemente agrupando N Flip-Flops (usualmente tipo D), compartiendo la misma señal de reloj. Permite almacenar temporalmente una "palabra" binaria de N bits.

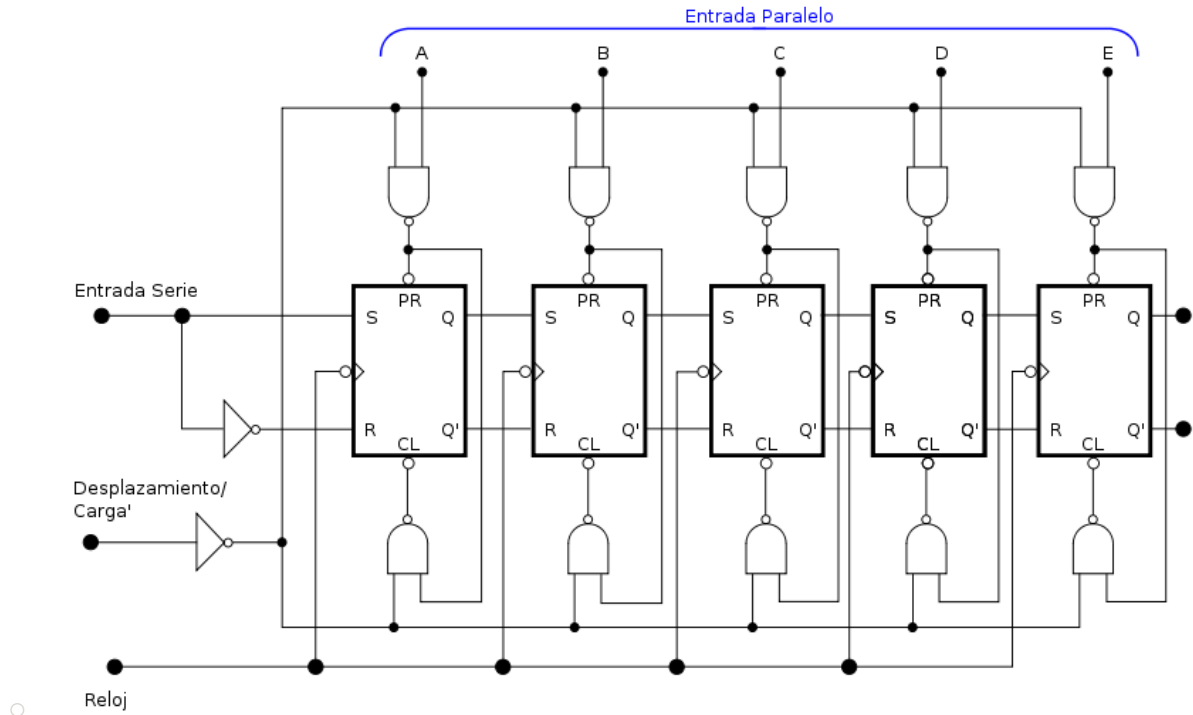
- **Carga Paralelo:** Si cada entrada D tiene su propia línea de datos, podemos cargar los N bits simultáneamente con un pulso de reloj. *Por qué:* Almacenamiento rápido de datos (ej: resultado de una operación en la CPU).
- **Registros de Desplazamiento (Shift Registers): Moviendo Bits:**
 - *Cómo se conectan:* La salida Q de un FF se conecta a la entrada D del siguiente FF de la cadena. Todos comparten el mismo CLK.
 - *Funcionamiento:* Con cada pulso de reloj, el bit de cada FF se "desplaza" a la posición del siguiente FF. Un bit nuevo entra por el primer FF y el último bit sale (o se pierde).
 - *Tipos y Aplicaciones ("Por qué" son útiles):*
 - **SISO (Serial In, Serial Out):** Entra 1 bit, sale 1 bit por pulso. *Aplicación:* Generar retardos de tiempo digitales (N pulsos para que un bit recorra N FFs).



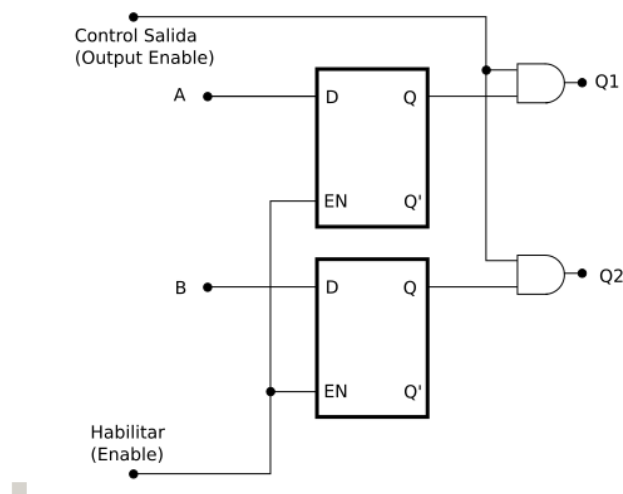
- **SIPO (Serial In, Parallel Out):** Entran los bits uno a uno (en serie) pero se pueden leer todos los bits almacenados a la vez (en paralelo) desde las salidas Q de cada FF. *Aplicación:* Convertir datos recibidos en serie (ej: de un sensor, de una comunicación) a formato paralelo para que la CPU/MCU los procese más rápido.



- **PISO (Parallel In, Serial Out):** Se cargan todos los bits a la vez (en paralelo) y luego se extraen uno a uno por la salida del último FF con pulsos de reloj. *Aplicación:* Convertir datos en paralelo (ej: de un puerto de entrada) a formato serie para transmitirlos por un solo cable.



- **PIPO (Parallel In, Parallel Out):** Carga y lectura en paralelo. Es básicamente el registro de almacenamiento que vimos antes.



- **Memorias Semiconductoras: Almacenamiento Masivo y Organizado:**

- *Por qué:* Necesitamos guardar grandes cantidades de datos y programas (miles o millones de bits). Los registros son demasiado costosos y ocupan mucho espacio para esto.
- **RAM (Random Access Memory - Memoria de Acceso Aleatorio):**
 - *Características clave:* Permite **Leer y Escribir** datos rápidamente. Es **Volátil** (pierde la información si se corta la alimentación). Acceso aleatorio significa que se puede acceder a cualquier ubicación (dirección) directamente, sin pasar por las anteriores, y el tiempo de acceso es prácticamente el mismo para todas.
 - *Tipos (Cómo funcionan conceptualmente):*
 - **SRAM (Static RAM):** Cada bit se almacena en un pequeño circuito biestable (similar a un Latch/FF). *Ventajas:* Muy rápida, no necesita refresco. *Desventajas:* Menos densa (más transistores por bit), más cara, mayor consumo estático. *Uso:* Memoria Caché (memoria ultra-rápida dentro o cerca de la CPU).

- **DRAM (Dynamic RAM):** Cada bit se almacena como una pequeña carga eléctrica en un capacitor asociado a un transistor. *Ventajas:* Muy densa (menos componentes por bit), más barata. *Desventajas:* Más lenta que SRAM, la carga del capacitor se fuga y necesita ser **refrescada** periódicamente (leída y reescrita) para no perder datos, mayor consumo dinámico (durante lectura/escritura/refresco). *Uso:* Memoria principal de las computadoras y muchos sistemas embebidos.
- **ROM (Read Only Memory - Memoria de Solo Lectura):**
 - **Característica clave: No Volátil** (conserva la información sin alimentación). Originalmente sólo permitía lectura, pero las variantes modernas permiten escritura (aunque más lenta o compleja que la RAM).
 - **Tipos y Evolución (Cómo se graban/borran):**
 - **ROM (Mask ROM):** Grabada permanentemente durante la fabricación del chip. No se puede modificar.
 - **PROM (Programmable ROM):** Se compra "virgen" y el usuario la puede programar eléctricamente *una sola vez* (quemando "fusibles" internos).
 - **EPROM (Erasable PROM):** Se puede borrar exponiéndola a luz ultravioleta intensa (a través de una ventanita de cuarzo en el chip) y luego reprogramar eléctricamente. Obsoleta.
 - **EEPROM (Electrically Erasable PROM):** Se puede borrar y reprogramar eléctricamente byte por byte, directamente en el circuito. Más lenta y con un número limitado de ciclos de escritura.
 - **Memoria Flash:** Similar a EEPROM pero se borra en bloques más grandes (no byte a byte), es más rápida y densa. Es la tecnología usada en pendrives, tarjetas SD, SSDs y para almacenar el firmware en la mayoría de MCUs y dispositivos modernos.
 - **Por qué es crucial:** Almacena el programa inicial que ejecuta un sistema al encenderse (BIOS en PCs, Bootloader en MCUs), firmware de dispositivos, tablas de datos permanentes.
- **Organización y Acceso a Memoria:**
 - **Cómo se organiza:** Como un gran conjunto de "casilleros" o "celdas", cada una con una **dirección** única (un número binario que la identifica) y capaz de almacenar un dato (usualmente 8 bits = 1 Byte, o más). La capacidad se expresa como (Número de direcciones) x (Tamaño del dato), ej: 2K x 8 significa 2048 direcciones, cada una guarda 8 bits.
 - **Cómo se accede:** Se usa un **Bus de Direcciones** para seleccionar la celda deseada (la CPU/MCU pone la dirección en este bus). Se usa un **Bus de Datos** bidireccional para transferir el dato (leerlo desde la celda o escribirlo hacia ella). Se usan **Señales de Control** (ej: Chip Select/Enable para activar el chip de memoria, Read/Write para indicar la operación) para coordinar la transferencia. *Por qué buses compartidos:* Para reducir el número de pines y conexiones.

5. Uniendo las Piezas: CPU, MCU y PLC (El "Cerebro" Digital)

Ya tenemos los ladrillos: lógica (compuertas), memoria de 1 bit (FFs), almacenamiento (registros, RAM, ROM). Ahora vemos cómo se organizan para crear unidades que procesan información y controlan sistemas.

- **CPU (Central Processing Unit - Unidad Central de Proceso): El Ejecutor de Instrucciones**

- Es el "cerebro" de cualquier computadora o sistema programable. Su función es buscar, decodificar y ejecutar instrucciones almacenadas en la memoria.
- *Arquitectura Von Neumann (Simplificada):* Es el modelo más común. Componentes clave y su función (*Cómo se relacionan con lo visto*):
 1. **Unidad Aritmético-Lógica (ALU):** Realiza las operaciones matemáticas (suma, resta) y lógicas (AND, OR, XOR, NOT, desplazamientos) sobre los datos. (*Está hecha con circuitos combinacionales basados en compuertas*).
 2. **Unidad de Control (UC):** Dirige todo el proceso. Genera las señales de control internas y externas (a memoria, E/S) para buscar la instrucción, decodificarla (entender qué hay que hacer), traer los operandos necesarios (datos) y ordenar a la ALU u otra unidad que realice la operación. (*Es un complejo circuito secuencial, usa FFs, contadores, decodificadores*). Incluye registros importantes como el Contador de Programa (PC - apunta a la próxima instrucción) y el Registro de Instrucción (IR - guarda la instrucción actual).
 3. **Registros Internos:** Pequeñas memorias ultra-rápidas *dentro* de la CPU para guardar temporalmente datos que se están usando frecuentemente (operandos, resultados intermedios, estado del sistema - flags). (*Están hechos con Flip-Flops*).
 4. **Buses:** Conjuntos de líneas que conectan los componentes: Bus de Datos (mueve datos/instrucciones), Bus de Direcciones (selecciona ubicación en memoria o E/S), Bus de Control (transporta señales de sincronismo y comando).
- *Ciclo Básico de Instrucción (Fetch-Decode-Execute):*
 1. **Fetch (Búsqueda):** La UC lee la dirección del PC, la pone en el bus de direcciones, pide una lectura a memoria y trae la instrucción al IR. Incrementa el PC.
 2. **Decode (Decodificación):** La UC analiza el código de la instrucción en el IR para saber qué operación realizar y dónde están los operandos.
 3. **Execute (Ejecución):** La UC genera las señales para que la ALU u otra unidad realice la operación, usando datos de registros o memoria, y guardando el resultado. (Este paso puede involucrar sub-ciclos para traer operandos, etc.). Y vuelve a empezar...

- **Microcontrolador (MCU - MicroController Unit): El Sistema Embebido en un Chip**

- *¿Qué es?* Es básicamente una pequeña computadora completa *en un solo circuito integrado*. Contiene no solo la CPU, sino también la memoria y los periféricos necesarios para interactuar con el mundo exterior.
- *¿Por qué se usan tanto?* Son baratos, pequeños, de bajo consumo y tienen todo lo necesario para controlar un dispositivo específico (un electrodoméstico, un sensor inteligente, un robot simple, etc.). Son el corazón de los "sistemas embebidos". La placa Arduino es un ejemplo popular de plataforma basada en un MCU.
- *Componentes Típicos Integrados (Además de CPU, RAM y ROM/Flash):*
 1. **Puertos de Entrada/Salida Digital (GPIO - General Purpose I/O):** Pines que se pueden configurar como entrada (leer un pulsador, un sensor digital) o como salida (encender un LED, activar un relé). *Cómo:* Usan registros internos para configurar dirección y leer/escribir el estado.

2. **Conversor Analógico-Digital (ADC):** Permite leer señales analógicas (ej: de un potenciómetro, un sensor de temperatura LM35) y convertirlas a un valor digital que la CPU pueda procesar. *Por qué:* El mundo real es mayormente analógico.
 3. **Conversor Digital-Analógico (DAC) / Modulación por Ancho de Pulso (PWM):** Permiten generar señales analógicas o pseudo-analógicas (PWM) para controlar actuadores (ej: brillo de un LED, velocidad de un motor DC).
 4. **Timers/Contadores:** Circuitos especializados para medir tiempo, contar eventos externos, generar señales periódicas (como el PWM) o disparar interrupciones después de cierto tiempo. *Por qué:* Liberan a la CPU de tareas de temporización.
 5. **Interfaces de Comunicación Serial:** Para hablar con otros dispositivos: UART (comunicación asíncrona simple, ej: con PC, GPS), SPI e I2C (comunicación síncrona con sensores, memorias externas, otros chips).
- **Controlador Lógico Programable (PLC - Programmable Logic Controller): El Estándar Industrial**
 - *¿Qué es y por qué es diferente?* Es un tipo de computadora industrial diseñada específicamente para sobrevivir y operar de forma fiable en entornos industriales hostiles (ruido eléctrico, vibraciones, temperaturas extremas) y para controlar procesos en tiempo real. Su diseño prioriza la **robustez**, la **facilidad de conexión** de E/S industriales y la **programación intuitiva** para técnicos (lenguaje Escalera/Ladder).
 - *Arquitectura Típica:*
 1. **CPU Industrial:** Optimizada para lógica de control y manejo rápido de E/S.
 2. **Memorias:** RAM (para datos temporales/programa de usuario), ROM/Flash (para firmware/sistema operativo del PLC). A veces con batería de respaldo para la RAM.
 3. **Módulos de Entrada (Digitales/Analógicas):** Toman las señales del campo (sensores, pulsadores - ej: 24VDC, 110VAC) y las *aíslan ópticamente* (optoacopladores) y *acondicionan* para proteger la CPU y convertirlas a niveles lógicos internos.
 4. **Módulos de Salida (Digitales/Analógicas):** Toman las señales lógicas de la CPU y las convierten para activar actuadores de campo (contactores, electroválvulas, variadores). Pueden ser a Relé (contactos secos, versátiles pero lentos y con desgaste) o a Transistor (rápidos, sin desgaste, pero para DC y menor corriente). También aíslan ópticamente. Las salidas analógicas generan señales estándar (4-20mA, 0-10V).
 5. **Fuente de Alimentación Industrial:** Robusta, filtrada, a menudo de 24VDC.
 6. **Bus de Sistema:** Conecta todos los módulos (si es modular).
 7. **Puerto de Programación:** Para conectar una PC o consola y cargar/modificar el programa.
 - **Ciclo de Scan (Cómo opera):** A diferencia de una PC, el PLC ejecuta un ciclo repetitivo muy rápido:
 1. **Leer Entradas:** Lee el estado de todas las entradas físicas y lo guarda en una memoria interna (Imagen de Entradas).
 2. **Ejecutar Programa:** Ejecuta la lógica programada por el usuario (ej: Ladder) utilizando la Imagen de Entradas y datos internos. Calcula el estado de las salidas y lo guarda en otra memoria (Imagen de Salidas).
 3. **Tareas Internas:** Realiza autodiagnósticos, comunicaciones, etc.

4. **Actualizar Salidas:** Transfiere el estado de la Imagen de Salidas a las salidas físicas. Y vuelve a empezar... *Por qué este ciclo:* Es determinista (se sabe cuánto tarda aproximadamente) y asegura una respuesta predecible del sistema de control.

- **Ventajas Clave en Industria:** Robustez física y eléctrica, modularidad (fácil expansión/reparación), lenguajes de programación estándar (Ladder es muy visual para eléctricos/electromecánicos), facilidad de diagnóstico y mantenimiento.

- **Integración Final:** Es vital entender que la CPU, el MCU y el PLC, aunque diferentes en su aplicación y robustez, internamente se basan en los mismos principios de la electrónica digital que estudiamos: la ALU realiza operaciones con compuertas, la Unidad de Control es lógica secuencial compleja basada en FFs, los registros usan FFs, el programa y los datos se almacenan en memorias (RAM/ROM) construidas con tecnología de FFs o capacitores/transistores. Dominar los fundamentos digitales nos permite entender el corazón de cualquier sistema de control moderno.

Síntesis (1 oración): La electrónica digital, desde el transistor como interruptor hasta la arquitectura de CPUs, MCUs y PLCs, proporciona las herramientas fundamentales para procesar información binaria, almacenar estados y tomar decisiones lógicas, siendo la base tecnológica del control automático moderno en sistemas electromecánicos.

Síntesis (5 puntos clave):

1. **Base Digital:** El transistor en corte/saturación representa los estados binarios 0/1, y las compuertas lógicas (AND, OR, NOT, etc.) realizan operaciones básicas sobre ellos.
2. **Álgebra de Boole:** Permite describir y simplificar matemáticamente los circuitos lógicos combinacionales, optimizando su diseño (costo, velocidad).
3. **Lógica Secuencial (Flip-Flops):** Introducen la memoria (1 bit por FF - JK, D, T), permitiendo a los circuitos "recordar" estados pasados y operar de forma sincronizada con un reloj.
4. **Almacenamiento (Registros y Memorias):** Los registros (grupos de FFs) guardan/mueven palabras binarias; las memorias RAM (volátil, R/W) y ROM (no volátil, lectura ppal.) almacenan grandes cantidades de datos y programas usando direcciones.
5. **Unidades de Procesamiento (CPU/MCU/PLC):** Integran lógica combinacional (ALU), secuencial (UC, registros) y memoria para ejecutar instrucciones (CPU), controlar sistemas embebidos (MCU) o procesos industriales robustamente (PLC), aplicando los principios digitales estudiados.